

CodeClash.AI

**A PROJECT REPORT
for
Major Project-I (CA301P)
Session (2025-26)**

Submitted by

**Mohammad Daud
202410116100121**

**Mayank Srivastava
202410116100118**

**Submitted in partial fulfilment of the
Requirements for the Degree of**

MASTER OF COMPUTER APPLICATION

**Under the Supervision of
Dr. Ankit Verma
Associate Professor
Ms. Shruti Aggarwal
Assistant Professor**



Submitted to

**DEPARTMENT OF COMPUTER APPLICATIONS
KIET Group of Institutions, Ghaziabad
Uttar Pradesh-201206
(NOV - 2025)**

Declaration

We, the undersigned, hereby declare that the Mini Project titled "**CodeClash.AI**" is an original work completed by us as part of the curriculum requirement for the course under the Master of Computer Applications (MCA) program at **KIET Group of Institutions**.

We affirm that we have undertaken this during the academic year 2025-26 under the guidance of **Dr. Ankit Verma**. All the content and ideas presented in this report are the result of our own efforts, except where explicitly stated otherwise. Proper citations have been provided wherever references to external sources have been made.

We further declare that this project has not been submitted, either in part or in full, to any other university or institution for any degree or diploma.

Mohammad Daud

Mayank Srivastava

CERTIFICATE

Certified that **MOHAMMAD DAUD (202410116100121), MAYANK SRIVASTAVA (202410116100118)** have carried out the project work having “**CodeClash.AI**” (**Major Project-I, CA301P**) for **Master of Computer Application** from Dr. A.P.J. Abdul Kalam Technical University (AKTU) (formerly UPTU), Lucknow under my supervision. The project report embodies original work, and studies are carried out by the student himself, and the contents of the project report do not form the basis for the award of any other degree to the candidate or to anybody else from this or any other University/Institution.

Dr. Ankit Verma

Associate Professor

Department of Computer Applications

KIET Group of Institutions, Ghaziabad

Dr. Sachin Malhotra

Dean

Department of Computer Applications

KIET Group of Institutions, Ghaziabad

Abstract

The CodeClash.AI is a web application designed for students, which is an educational platform where user can do competitive coding along with variety of questions. Along with coding, users can do quiz with choice of questions where difficulty level is low to high and once submitting the quiz user can check his performance and also get feedback. In this project we address some insufficiencies about previously developed quiz platforms and with this we make this interactive and knowledge based. This project addresses inefficiencies in traditional educational model and ensures accuracy and quality education.

Key features of the system include:

- A user-friendly interface built using React and Tailwind CSS.
- Toasts are used to make interface interactive.
- Create a smooth backend using NodeJS, ExpressJS (JavaScript).
- MongoDB is used for backend.
- Focused on quality education, feels like a game.
- Secure and scalable architecture for efficient performance.

This project demonstrates the use of modern web technologies to enhance administrative workflows in educational institutions and help the students and competitive programmers to enhance programming skills with problem solving.

Keywords: CodeClash.AI, coding, React, Tailwind CSS, Node, JavaScript

Acknowledgments

Success in life is never attained single-handedly. My deepest gratitude goes to my project supervisor **Dr. Ankit Verma**, for her guidance, help, and encouragement throughout my project work. Their enlightening ideas, comments, and suggestions. Words are not enough to express my gratitude to **Dr. Sachin Malhotra**, Professor and Dean, Department of Computer Applications, for his insightful comments and administrative help on various occasions. Fortunately, I have many understanding friends, who have helped me a lot on many critical conditions. Finally, my sincere thanks go to my family members and all those who have directly and indirectly provided me with moral support and other kinds of help. Without their support, completion of this work would not have been possible in time. They keep my life filled with enjoyment and happiness.

Name – Mayank Srivastava

Roll No - 202410116100118

Name - Mohammad Daud

Roll No - 202410116100121

TABLE OF CONTENTS

	Declaration.....	i
	Certificate.....	ii
	Abstract.....	iii
	Acknowledgments.....	iv
	Table of Contents.....	v
1	Introduction.....	7-9
	1.1 Overview.....	7
	1.2 Basic Communication Model.....	7
	1.3 Data Communication... ..	8
2	Literature Review.....	10-13
	2.1 Key features of CodeClash.AI.....	10-11
	2.2 Role of modern web technologies.....	12-13
3	Project Flow.....	14-16
	3.1 Use Case Diagram.....	14
	3.2 Data Flow Diagram.....	14-15
	3.3 ER Diagram.....	16
4	Database Tables.....	17-19
5	System Structure.....	20-21
	5.1 System Overview	20
	5.2 System Architecture.....	20-21
6	Implementation.....	22-27
7	Project Outcome	28-29
8	Testing.....	30-31

9	Project Output.....	32-33
10	Output Screenshots.....	34-38
11	Conclusion.....	39
12	References.....	40

LIST OF FIGURES

1	Project Flow.....	14
2	Use Case Diagram.....	14
3	Level 0 DFD.....	15
4	Level 1 DFD.....	15
5	Level 2 DFD.....	16
6	ER Diagram.....	16

1. Introduction

In today's era of artificial intelligence and cloud-based learning, traditional coding platforms such as LeetCode, CodeChef, and HackerRank have become benchmarks for competitive programming. However, these platforms often lack an integrated learning environment that helps students not only *solve* problems but also *understand* the underlying concepts, logic, and optimized approaches in an interactive and personalized manner.

CodeClash.AI bridges this gap by combining coding, learning, and artificial intelligence into one unified platform. It is an **AI-powered coding and learning system** designed for students who want to strengthen their programming fundamentals through hands-on problem-solving supported by an intelligent learning assistant. The platform enables users to code directly in the **Monaco Editor**, execute and test their solutions, and explore the **AI Learn** section powered by **Google's Gemini API** to receive explanations, step-by-step approaches, and optimized implementations for each problem.

This project aims to create a comprehensive environment where students can practice coding, learn from AI-generated insights, and build real-world problem-solving skills that align with industry standards. The platform also ensures secure data handling through a robust backend built with **Node.js and Express.js**, and uses **MongoDB** for efficient and scalable data storage.

Key Features:

- User Profiles – Register and manage your progress.
- Coding Challenges – Timed coding tasks with instant feedback.
- Quiz Section – Theory-based quizzes to boost conceptual understanding.
- Real-Time Evaluation – Scoring based on accuracy and speed.

1.1 Overview

CodeClash.AI is an AI-enhanced educational platform that combines the power of **interactive coding** with **intelligent learning support**. The system allows students to choose problems of varying difficulty, solve them within the web-based Monaco Code Editor, and analyze their solutions with detailed AI-guided feedback.

The AI module, powered by the **Gemini API**, interprets user queries, explains logic and edge cases, and suggests improvements to their code. This ensures not just rote coding practice but conceptual learning and real-time problem-solving guidance.

A good system must meet the following criteria:

- **Performance:** Fast, efficient code execution and AI response handling.
- **Reliability:** Stable backend services and consistent AI interactions.
- **Scalability:** Ability to support multiple users and concurrent AI requests.
- **Security:** Encrypted user authentication and safe data communication between components.

1.2 Basic Communication Model

CodeClash.AI operates on a **client–server model** integrated with an AI communication layer.

- The **frontend**, developed in **ReactJS**, manages user interactions, including problem selection, code execution, and AI queries through a clean, responsive interface.
- The **backend**, powered by **Node.js** and **Express.js**, handles request routing, authentication, and communication with both the MongoDB database and the Gemini API.
- The **database layer** uses **MongoDB** to store user profiles, code submissions, problem statements, and AI-learning session data.
- The **AI module**, powered by **Gemini**, processes user requests such as “explain this problem” or “optimize my solution,” and returns detailed explanations, approaches, and code samples through secure API calls.

This multi-tier architecture ensures smooth data flow and reliable communication between all components.

1.3 Data Communication

The **data communication model** of CodeClash.AI ensures efficient request–response handling between the client, server, and AI layer.

1. The **frontend (ReactJS)** sends API requests to the **Node.js backend** for operations such as user authentication, fetching questions, and submitting code.
2. When the user invokes the **AI Learn** feature, the backend securely forwards the user's code and problem statement to the **Gemini API**, which processes the input and returns structured AI responses.
3. The **backend** then parses the AI output, reformats it into a readable editorial (approach + explanation + code), and sends it back to the frontend.
4. **MongoDB** stores relevant session data, including the user's attempts, performance metrics, and learning interactions.

This communication model ensures seamless interaction between the student, the platform, and the AI assistant—offering an intelligent, responsive, and secure coding environment.

2. Literature Review

In the current digital education ecosystem, a variety of online coding platforms exist to help learners develop programming skills. Websites like **LeetCode**, **HackerRank**, and **CodeChef** provide structured problem sets and competitive environments. However, these platforms primarily focus on problem-solving without integrating *intelligent guidance or contextual learning support*. Learners often struggle to understand why a particular approach works, or how to optimize their code efficiently.

CodeClash.AI distinguishes itself from these traditional platforms by introducing an **AI-driven learning model**. It integrates artificial intelligence to assist students not only in solving problems but also in *understanding the logic, approach, and best coding practices*. Using the **Gemini API**, CodeClash.AI generates AI-powered editorials, algorithmic breakdowns, and optimized implementations.

Thus, CodeClash.AI acts as a **complete learning assistant**, combining the best aspects of interactive coding, intelligent feedback, and conceptual understanding — all within a seamless, web-based environment powered by modern technologies.

2.1 Key Features of CodeClash.AI

1. AI-Learn Assistant (Powered by Gemini API)

A built-in AI system provides students with intelligent support, including problem explanations, editorial guidance, and step-by-step solutions. When users click on the “AI Learn” option in the code editor, Gemini analyzes their query and generates:

- Problem breakdown and explanation.
- Step-by-step solution approach.
- Optimized code in the preferred language.
- Comparison of user code with AI-generated code for learning improvement.

2. Integrated Monaco Code Editor

The platform uses the **Monaco Editor** — the same code editor used in Visual Studio Code — allowing syntax highlighting, auto-completion, and live error detection. This provides a real-world coding environment directly in the browser.

3. Intelligent Feedback and Learning Analytics

User submissions are analyzed, and performance metrics such as accuracy, number of attempts, and time taken are stored. This helps in tracking learning progress and provides insights into areas of improvement.

4. Interactive Coding Experience

Users can select problems categorized by difficulty (Easy, Medium, Hard) and topic (Data Structures, Algorithms, Strings, etc.), code directly within the browser, and execute their solutions.

5. Scalable and Secure Backend

The backend, developed with **Node.js** and **Express.js**, ensures smooth communication between the frontend, database, and AI service. It uses **JWT-based authentication** for secure access and **MongoDB Atlas** for scalable data storage.

6. Personalized Learning

The platform stores each user's interaction with the AI assistant and problem-solving history, which can be used to generate adaptive recommendations for further practice and learning in the future.

7. Modern UI with ReactJS

A responsive and dynamic user interface developed using **ReactJS** ensures smooth navigation and an engaging user experience. The AI learn section, code editor, and output panels are neatly organized for ease of use.

2.2 Role of Modern Web Technologies

CodeClash.AI leverages a combination of **frontend, backend, AI, and database technologies** to create a secure, scalable, and intelligent learning platform.

1. Frontend – ReactJS + Monaco Editor

The frontend of CodeClash.AI is built using **ReactJS**, which allows component-based architecture and dynamic rendering for an interactive experience.

The **Monaco Code Editor** is embedded within React to provide a VS Code–like environment directly in the browser.

Key Advantages:

- Syntax highlighting and intelligent code suggestions.
- Customizable themes and layouts.
- Real-time execution and output rendering.

2. Backend – Node.js and Express.js

The backend is responsible for routing, business logic, API communication, and AI service integration. It handles:

- User registration and authentication via **JWT tokens**.
- Routing requests to the Gemini AI service and MongoDB.
- Managing user submissions and AI-learn sessions.
- Serving the frontend data through REST APIs.

Why Node.js & Express.js?

- Asynchronous and event-driven nature suitable for handling multiple simultaneous users.
- Fast I/O operations using non-blocking architecture.
- Easy integration with AI APIs and MongoDB.

3. Database – MongoDB

A NoSQL database, **MongoDB**, stores flexible and dynamic data such as:

- User profiles and authentication data.
- Problem statements and metadata.

- Code submissions, AI-learn responses, and progress records.

Advantages:

- High scalability for concurrent user handling.
- JSON-like document structure aligning well with Node.js.
- Fast query performance for real-time updates.

4. AI Layer – Gemini API

The Gemini API forms the core of the intelligent learning module. It processes requests from the backend to generate:

- Editorials and solution explanations.
- Algorithmic insights and code optimizations.
- Conceptual breakdowns for better understanding.

Benefits of AI Integration:

- Real-time assistance without leaving the coding environment.
- Adaptive learning that responds to user queries.
- Enhanced engagement through instant knowledge retrieval.

5. Authentication & Security

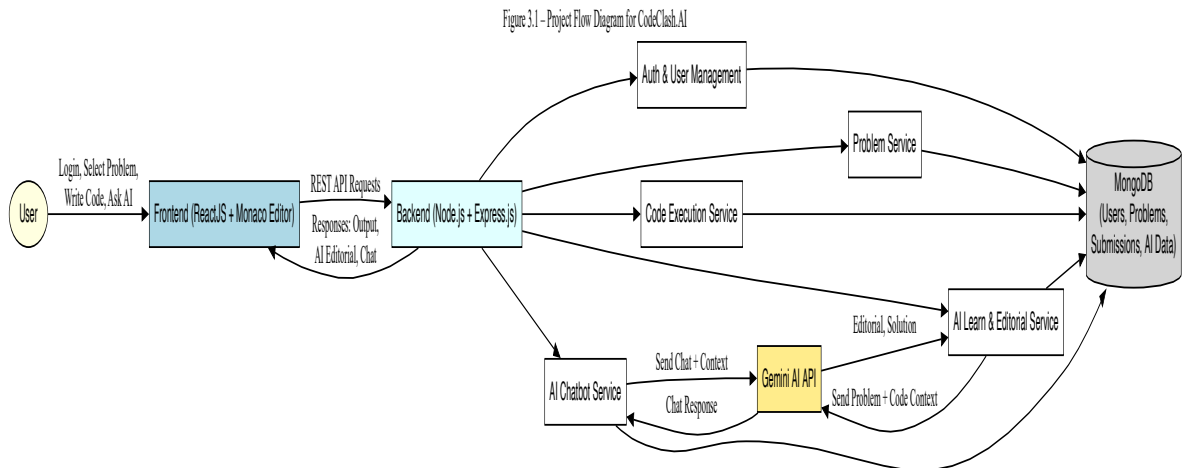
Security is implemented using:

- **JWT (JSON Web Tokens)** for safe user sessions.
- **Password hashing and encryption** using bcrypt.
- **CORS configuration** to prevent unauthorized API access.

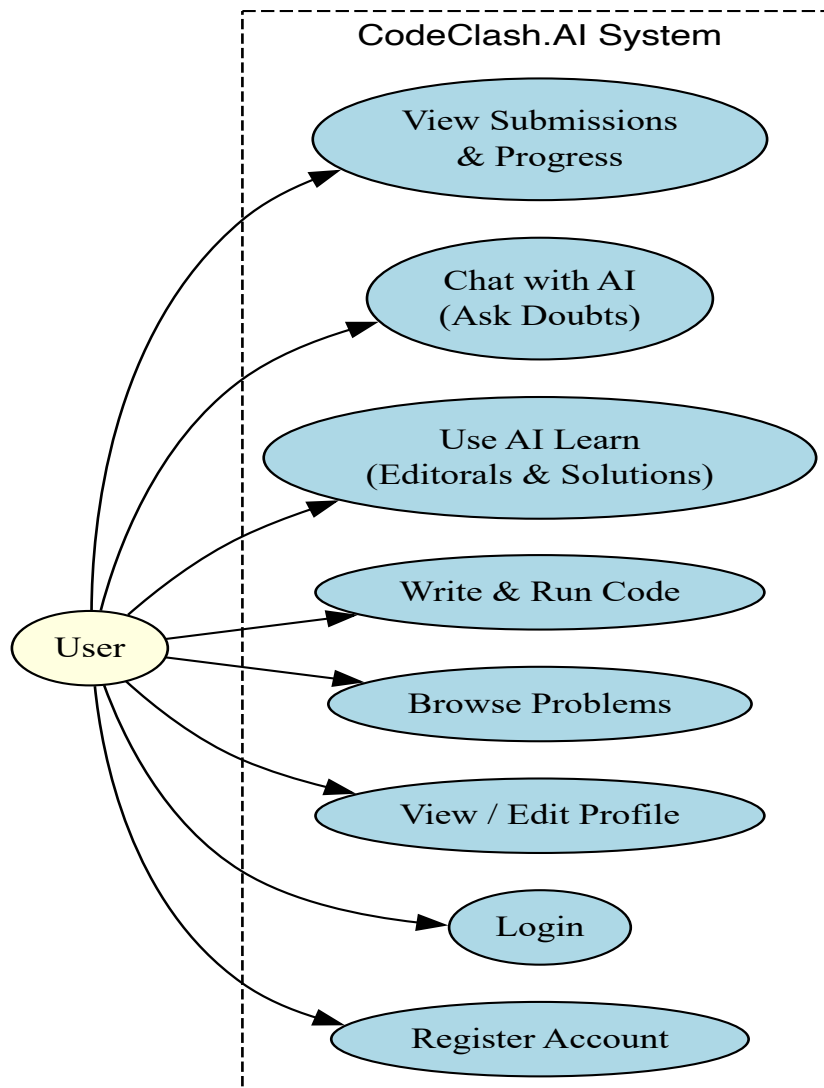
Cloud Integration & Scalability

- **MongoDB Atlas** ensures globally distributed, scalable cloud data storage.
- Backend hosted on **Node.js environment (e.g., Render/Heroku)** ensures automatic scaling with user load.

3. Project Flow

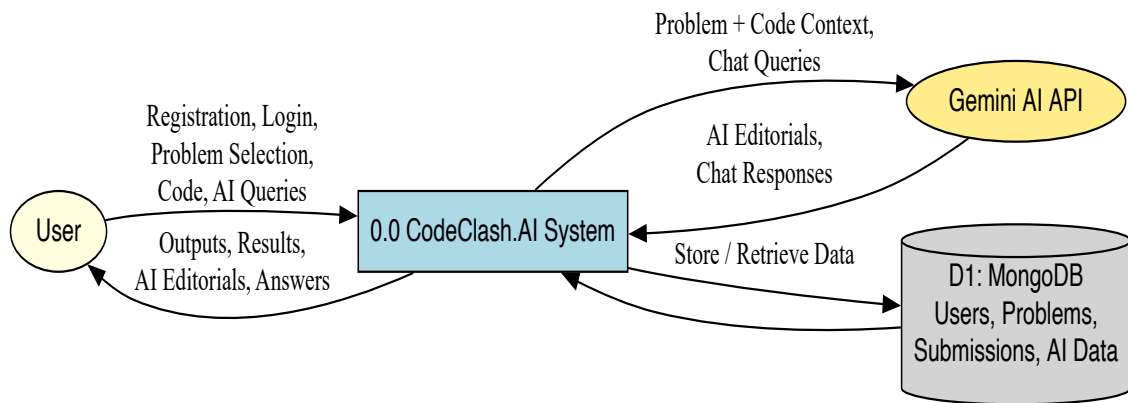


3.1 High Level Project Flow Diagram



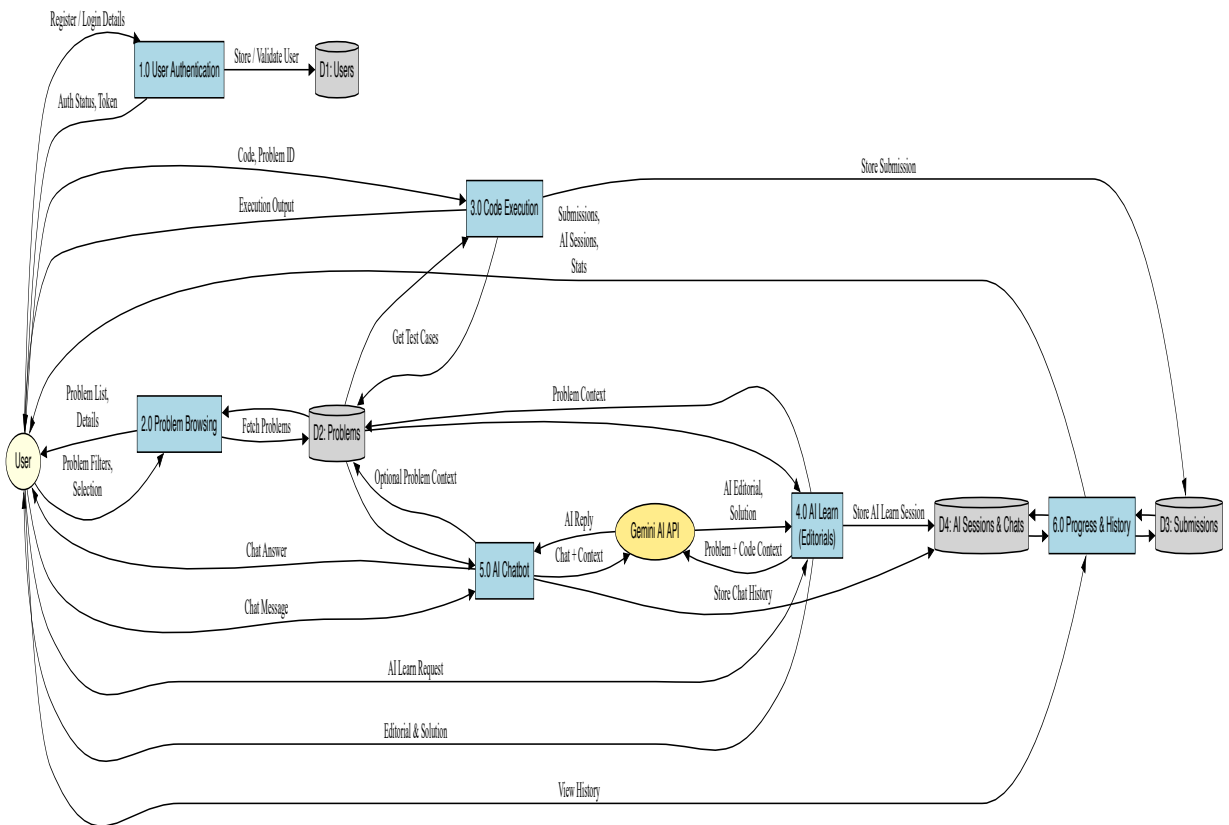
3.2 Use Case Diagram

Figure 3.3 – Level 0 DFD for CodeClash.AI



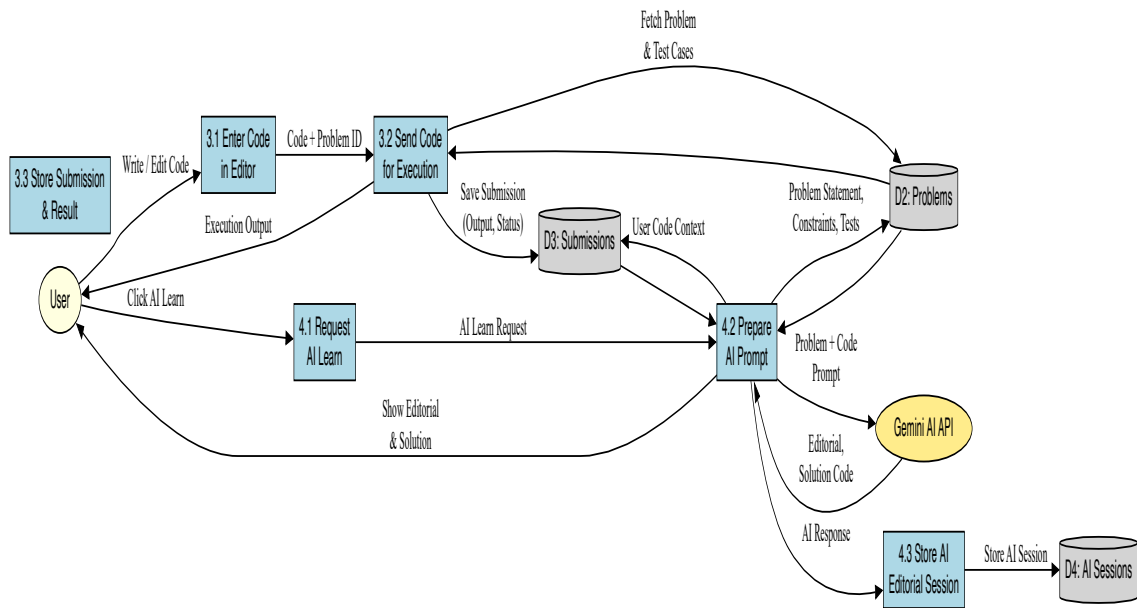
3.2 Level 0 DFD

Figure 3.4 – Level 1 DFD for CodeClash.AI



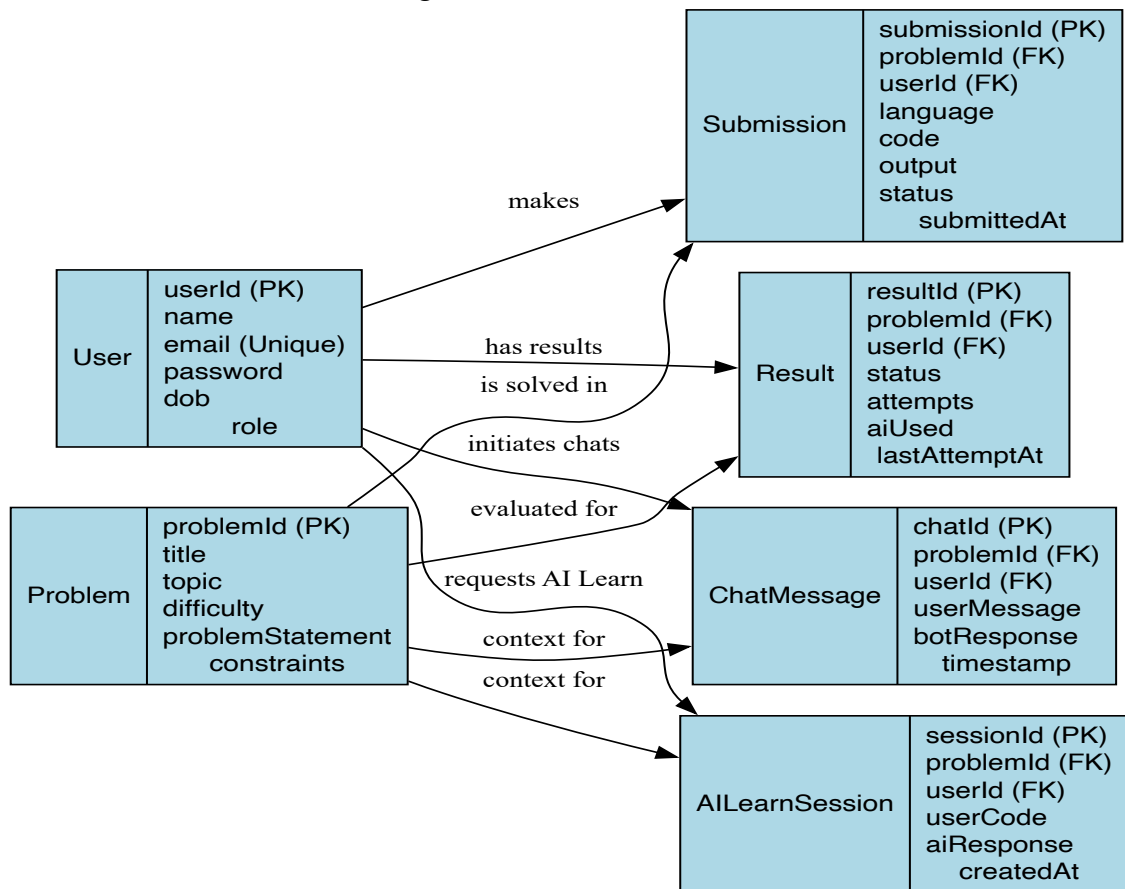
3.3 Level 1 DFD

Figure 3.5 – Level 2 DFD for Code Execution & AI Learn



3.4 Level 2 DFD

ER Diagram – CodeClash.AI Data Model



3.5 ER Diagram

4. Database Tables for CodeClash.AI

4.1 Users Collection

Field Name	Type	Description
id (userId)	ObjectId (PK)	Unique identifier for each user
name	String	Full name of the user
email	String (Unique)	User's email used for login
password	String	Hashed password (bcrypt encrypted)
role	String	User role: STUDENT / ADMIN
dob	Date	Date of birth of the user
createdAt	Timestamp	Date and time of account creation

4.2 Questions Collection

Field Name	Type	Description
id (questionId)	ObjectId (PK)	Unique identifier for the coding problem
title	String	Short title of the problem
problemStatement	Text	Detailed description of the problem
difficulty	String	Difficulty level (Easy / Medium / Hard)
topic	String	Topic category (Arrays, Strings, DP, etc.)
sampleInput	String	Sample input for understanding
sampleOutput	String	Expected output for sample input
constraints	String	Constraints related to the problem
testCases	Array	List of input-output pairs for evaluation

4.3 Solution Submissions Collection

Field Name	Type	Description
_id (submissionId)	ObjectId (PK)	Unique identifier for each submission
problemId	ObjectId (FK)	Reference to the solved problem
email	String (FK)	Email of the user who submitted the solution
language	String	Programming language used
code	Text	Submitted source code
output	String	Execution result (output or error)
status	String	Accepted / Wrong Answer / Error
submittedAt	Timestamp	Time of submission

4.4 AI Learn Sessions Collection

Field	Type	Description
_id (sessionId)	ObjectId (PK)	Unique AI learning session identifier
email	String (FK)	User who requested AI assistance
problemId	ObjectId (FK)	Problem for which AI help was requested
userCode	Text	User's code sent for explanation
aiResponse	Text	Gemini-generated explanation and solution
createdAt	Timestamp	Time of AI Learn request
testCases	Array	List of test case input-output pairs for evaluation.

4.5 Chatbot Conversations Collection

Field	Type	Description
_id (chatId)	ObjectId (PK)	Unique chatbot conversation ID
email	String (FK)	User who interacted with the chatbot
problemId	ObjectId (FK)	Associated problem context
userMessage	Text	User's chatbot query
botResponse	Text	AI chatbot reply
timestamp	Timestamp	Time of interaction

4.6 Results Collection

Field	Type	Description
_id (resultId)	ObjectId (PK)	Unique result record ID
email	String (FK)	User associated with the result
problemId	ObjectId (FK)	Problem attempted
status	String	Solved / Unsolved
attempts	Number	Number of attempts made
aiUsed	Boolean	Whether AI Learn or chatbot was used

5. System Structure

5.1 Overview

The **CodeClash.AI** system follows a **modular three-tier architecture** integrated with an **AI-driven learning and chatbot layer**. It is designed to provide a complete, interactive coding environment powered by artificial intelligence for conceptual understanding, instant editorial generation, and doubt resolution.

The system enables users to:

- Practice coding problems in real-time using an in-browser editor.
- Execute and test code outputs directly.
- Access the **AI Learn** feature to receive problem editorials, approaches, and optimized code via **Gemini API**.
- Interact with a **built-in AI chatbot** that reads the current problem context and answers queries related to the problem or user code.

The architecture emphasizes **interactivity**, **intelligence**, and **scalability** — providing a seamless experience that blends practical coding with adaptive AI learning.

5.2 System Architecture

The CodeClash.AI platform consists of the following layers:

5.2.1 Frontend Layer (Client-Side)

- Provides a rich, dynamic user interface where users can log in, select problems, write and execute code, and chat with the AI assistant.
- The **Monaco Editor** (used in VS Code) is integrated for a smooth coding experience with syntax highlighting, auto-completion, and live error detection.
- The **AI Chatbot** and **AI Learn** features are embedded directly in the coding environment, enabling continuous learning without leaving the page.

5.2.2 Backend Layer (Server-Side)

- Manages user authentication, problem retrieval, code submission, AI requests, and database interactions.
- Processes all communication between the frontend and the **Gemini API**, ensuring secure handling of user queries and AI responses.
- Handles execution of user-submitted code on the server and returns outputs or errors in real-time.

5.2.3 Database Layer

- Uses MongoDB for storing Users, Questions, Submissions, Results, AI-Learn Sessions, ChatHistory
- Provides scalability and fast retrieval of problem and user data.

5.2.4 AI Layer

- Gemini acts as the core intelligence engine for CodeClash.AI.
- Powers two major features:
 - **AI Learn:** When a user selects this option, Gemini reads the problem statement and provides a detailed **editorial explanation, step-by-step approach, and optimized code.**
 - **AI Chatbot:** A context-aware assistant embedded in the editor that reads the current problem and user code to answer questions such as “What is wrong with my code?” or “Explain this problem in simple terms.”

5.2.4 System Flow

- **User Authentication:** Users register or log in securely through the frontend. The backend validates credentials and issues a JWT token for session management.
- **Problem Selection & Code Editing:** The user selects a coding problem and writes code inside the **Monaco Editor**.
- **Code Execution:** Upon clicking *Run*, the code is sent to the backend via REST API. The backend executes it and returns the output or error in real-time.
- **AI Learn:** When the user activates *AI Learn*, the backend forwards the problem statement and code to the **Gemini API**, which generates an editorial explanation and optimized solution.
- **Chatbot Assistance:** The chatbot continuously monitors problem context and user input. It provides instant, conversational help related to the problem, code logic, or debugging.
- **Result Tracking:** Each user’s performance, AI interactions, and progress are recorded in **MongoDB** for analytics and personalized feedback.

5.2.5 Code Execution Service

- A server-side **secure sandbox** or isolated worker (containerized) executes user code against test cases and returns the output, runtime, and errors through REST.
- Execution is synchronous from the frontend point of view (user clicks Run → HTTP request → response with results).

6. Implementation

6.1 Phase 1: Frontend Development

Technologies Used:

- **ReactJS** – For modular and responsive component-based UI.
- **Monaco Editor** – For the in-browser coding environment.
- **Tailwind CSS** – For styling, responsiveness, and layout consistency.
- **Axios / Fetch API** – For sending HTTP requests to the backend.

Implementation Details:

1. User Interface Design

- A clean, minimal, and intuitive UI was built using ReactJS functional components.
- Key screens:
 - Login / Registration Page – For secure authentication.
 - Dashboard – Displays coding problems with filters for difficulty and topic.
 - Code Editor Page – Hosts the Monaco Editor, the AI Learn panel, and the AI Chatbot window.

2. Code Editor Integration

- The **Monaco Editor** was embedded inside a dedicated React Component.
- Syntax highlighting, auto-completion, and live linting features were enabled for user productivity.
- A **Run** button triggers a POST request to the API endpoint with the user's code and selected problem ID.

2. AI Learn Integration

- A dedicated AI Learn button invokes the Gemini API.
- The frontend sends { problemId, userCode } to /api/ai/learn.
- The response is displayed inside an **AI Learn Drawer/Panel**, showing:
 1. Problem Editorial
 2. Approach Summary
 3. Optimized Solution Code

3. AI Chatbot Interface

- A chat window component was built similar to a messaging interface.
- User queries are sent to /api/ai/chat along with the current problem context.
- The Gemini API response is displayed in real time as a chat bubble.

4. User Experience Features

- **Toast notifications** for execution status and errors.
- **LocalStorage** to maintain session state and JWT token.
- **Responsive Design** for desktop and tablet use.

Phase 2: Backend Development

Technologies Used:

- **Node.js + Express.js** – REST API and middleware control.
- **MongoDB Atlas** – Data storage and querying.
- **JWT + Bcrypt** – Authentication and password encryption.
- **Docker-based execution worker** – For safe code execution.

Implementation Details:

1. API Structure

- The backend follows a modular Express routing structure:

backend/

```
|— index.js or server.js
|— routes/
|   |— gemini.js    ← AI routes (Gemini integration)
|   |— problems.js ← For coding problems
|   |— submissions.js ← For code execution results
|   |— users.js     ← For auth (login/register)
|— controllers/
|   |— geminiController.js
|   |— problemController.js
|   |— submissionController.js
|   |— userController.js
|— models/
|   |— User.js
|   |— Problem.js
|   |— Submission.js
|   |— ChatSession.js
|— utils/
|   |— codeExecutor.js
|   |— geminiAPI.js
```


2. Authentication

- Passwords are hashed using **bcrypt** before saving.
- JWT tokens are issued upon login and verified in every protected route.

3. Problem Management

- Problems are stored in MongoDB as documents containing title, difficulty, test cases, and constraints.
- `/api/problems` endpoint retrieves available problems for display.

4. Code Execution

- The `/api/execute` endpoint accepts `{ code, language, problemId }`.
- Code runs in a sandboxed environment via Docker or a child process

5. AI Learn and Chatbot Endpoints

- `/api/ai/learn` → Calls Gemini API for editorial & optimized solution.
- `/api/ai/chat` → Sends contextual prompt with problem and user code for a conversational response.
- All AI interactions are stored in **AI-Learn** and **ChatHistory** collections.

6. Data Persistence

- Each request outcome (login, submission, AI query) is stored in MongoDB with timestamps for audit and learning analytics.

Explanation:

Phase 1: Frontend Development

1 User Registration & Authentication

The user authentication interface was developed using **ReactJS**, with state management handled efficiently through **React Hooks**. The **Tailwind CSS** framework was used to design a clean, responsive, and modern UI for both the login and registration pages.

Upon successful authentication, a **JWT (JSON Web Token)** is issued by the backend and securely stored in the client's local storage. This token-based authentication approach eliminates the need for traditional server-side sessions and ensures scalability while maintaining security and data privacy.

2 Dashboard & Coding Environment

After logging in, each user is redirected to a **personalized dashboard** where they can browse coding problems categorized by difficulty and topic. The **Monaco Editor**, the same editor used in Visual Studio Code, is embedded in the coding page to provide a real-world programming experience within the browser.

The dashboard also integrates two major AI-powered features:

1. **AI Learn Panel** – Provides editorial explanations, algorithmic approaches, and optimized code solutions.
2. **AI Chatbot** – A context-aware assistant that allows students to ask problem-related or debugging questions in real time.

The frontend was designed using **component-driven development**, ensuring modularity and ease of maintenance. Only non-critical profile fields like name and date of birth are editable, while email and username remain locked to ensure account authenticity.

Phase 2: Backend Development

3 User Authentication & API Layer

The backend is developed using **Node.js** and **Express.js**, forming a lightweight and scalable RESTful architecture. Authentication is managed through **JWT tokens** and **bcrypt** encryption for password hashing.

Each user request is authenticated via middleware, where the JWT is validated before granting access to protected routes. This ensures a stateless and secure session handling mechanism without relying on server-side sessions.

4 Problem & Code Execution Management

The backend provides REST APIs for fetching coding problems, executing code, saving submissions, and retrieving user results. The **code execution module** uses a secure sandbox (via Docker or Node's `child_process`) to run user code in isolation and return the output or error response.

The earlier real-time quiz and room management features were completely removed to focus on **individual learning** and **AI-driven interactions**. The backend now serves as the communication bridge between the frontend, the database, and the **Gemini AI API**.

Phase 3: AI Integration and Chatbot Implementation

5 AI Learn System

The **AI Learn** module integrates the **Gemini API** to provide intelligent educational assistance. When a user activates AI Learn for a problem, the backend sends the problem statement and user's code to Gemini, which returns:

- A conceptual explanation of the problem,
- Step-by-step solution approach,
- Optimized code implementation in the desired language.

This feature promotes understanding of algorithms rather than rote memorization.

6 AI Chatbot

The **AI Chatbot**, also powered by Gemini, provides contextual, conversation-based assistance directly within the code editor. Users can ask natural-language queries such as:

- “Why am I getting a runtime error?”
- “Explain this problem in simple terms.”
- “What’s the time complexity of my approach?”

Each query is processed along with the current problem and code context, enabling highly relevant and adaptive responses. Chat history is stored in **MongoDB** for future learning analytics.

Phase 4: Testing & Deployment

7 Unit & Integration Testing

Comprehensive testing was conducted to validate both frontend and backend components. Each API endpoint — including authentication, problem retrieval, code execution, and AI responses — was thoroughly tested using **Postman** and **Jest**.

Integration testing ensured smooth coordination between modules, verifying complete user workflows such as:

- Login → Select Problem → Write Code → Run → Use AI Learn → Chat with AI → View Results.
- Edge cases (e.g., invalid tokens, compilation errors, and empty AI responses) were also handled gracefully.

8 Deployment

The final system was deployed on cloud platforms for scalability and easy access:

- **Frontend:** Deployed on **Vercel** for optimized performance and CI/CD integration.
- **Backend:** Hosted on **Render** or **Railway**, exposing RESTful endpoints securely.
- **Database:** Managed on **MongoDB Atlas**, providing global access and automatic backups.

Environment variables were used for all API keys (Gemini, JWT secrets), ensuring security during production deployment.

7. Project Outcome

The development of **CodeClash.AI** has resulted in a fully functional, AI-powered web platform that bridges the gap between **competitive programming** and **conceptual learning**. Unlike traditional coding practice websites, CodeClash.AI integrates an intelligent assistant that helps students understand the logic, approach, and optimization behind every problem they solve.

This project demonstrates how the combination of modern web technologies and artificial intelligence can create an **interactive, personalized, and educational coding experience**. It not only enhances programming skills but also develops critical thinking and problem-solving ability among students.

1. AI-Powered Learning Experience

The core innovation of CodeClash.AI lies in its Gemini-based AI integration, which allows students to receive:

- Step-by-step editorial explanations for each problem,
- Detailed algorithmic insights, and
- Optimized code solutions in their preferred programming language.

This eliminates the need for students to refer to multiple sources or discussion forums for learning — everything they need is available contextually within the platform.

2. Interactive In-Browser Coding Environment

The platform features an integrated Monaco Code Editor, offering users a real-time coding experience similar to Visual Studio Code. With syntax highlighting, auto-completion, and error detection, it ensures that learners can code efficiently without leaving the platform.

Users can write, run, and test their code instantly against pre-defined test cases, making the process both practical and engaging.

3. AI Chatbot for Contextual Assistance

An integrated AI Chatbot provides conversational assistance within the code editor.

It reads the current problem statement and user's code to offer personalized responses to queries such as:

- “Why is my output incorrect?”
- “Can you explain this algorithm?”

- “What are the edge cases for this problem?”

This feature transforms the platform into a virtual coding mentor, promoting self-paced and interactive learning.

4. Secure Authentication and Scalable Architecture

The use of JWT-based authentication ensures secure, stateless communication between the client and server. The system is designed with a MERN (MongoDB, Express.js, ReactJS, Node.js) architecture, making it modular, maintainable, and scalable for future expansion.

All user data, including submissions and AI interactions, are securely stored in MongoDB Atlas, providing persistent progress tracking and data integrity.

5. Data-Driven Learning Analytics

Every code submission, AI query, and learning interaction is logged for performance analytics.

This enables future enhancements such as:

- Personalized learning recommendations.
- Topic-wise progress tracking.
- Automated weak-topic detection through data trends.

These analytics can help educators and learners understand progress patterns and focus areas for improvement.

8. Testing

The testing phase of **CodeClash.AI** was a crucial stage in ensuring that the platform delivers a **secure, reliable, and seamless AI-powered coding experience**. Testing focused on verifying the correctness of all features — from authentication and code execution to AI-assisted learning and chatbot functionality.

A combination of **unit, integration, functional, performance, and security testing** was performed to ensure smooth operation across various modules and devices. The testing process also included validation of **Gemini API responses** to ensure accuracy and consistency in AI-generated explanations.

Testing Methodologies:

8.1 Unit Testing

- Unit testing was conducted to verify that individual modules and components of CodeClash.AI function correctly in isolation. Each module — including controllers, services, and utilities — was tested independently.

8.2 Integration Testing

- Integration testing ensured that all modules of the system — frontend, backend, database, and AI services — worked together cohesively.
- The focus was to validate the complete request–response cycle between user actions and system reactions.

8.3 Functional Testing

- Functional testing verified that **CodeClash.AI met all user requirements** and provided the intended learning experience. The testing process simulated real-world scenarios to ensure that users could navigate the system without encountering errors.
- Approach:

- Manual testing of the entire user journey, including registration, login, problem selection, code execution, AI Learn, and chatbot queries.
- Validation of AI responses for correctness, relevance, and formatting.
- Cross-browser and cross-device testing to ensure UI responsiveness.

8.4 Performance Testing

- Performance testing was carried out to ensure that CodeClash.AI handles concurrent users and multiple AI requests efficiently. The goal was to maintain quick response times for both code execution and AI responses.

8.5 Security Testing

- Security testing focused on ensuring data protection, secure communication, and controlled access to the system. Each component was tested to identify potential vulnerabilities that could compromise user data or system integrity.

8.6 Bug Fixing & Debugging Process

- During testing, several issues were identified and resolved to improve system stability and usability.

These included:

- Incorrect AI formatting in long responses (fixed via Markdown parser in frontend).
- Minor syntax highlighting lag in Monaco Editor (optimized through debounce).
- Duplicate submission records in MongoDB (resolved via unique constraints).
- CORS-related API request errors (fixed with middleware configuration).

8.7 Deployment Readiness

- After successful completion of all testing phases, **CodeClash.AI** was optimized for **cloud deployment and scalability**.
- Final verification was conducted across different devices (desktop, tablet) and browsers (Chrome, Edge, Firefox).

9. Project Output

Functional Output:

User Registration and Authentication:

Users can register and log in securely using **JWT-based authentication**. The frontend handles input validation while the backend verifies credentials and issues access tokens. Upon successful login, users gain access to protected routes such as the dashboard and coding environment.

Dashboard and Profile:

After logging in, users are directed to a personalized dashboard displaying their profile and progress. They can edit basic details like name and date of birth, while critical fields such as email and username remain locked for account integrity.

Problem Selection and Coding Interface:

Users can browse categorized coding problems based on difficulty and topic. Each problem includes a description, input/output format, and sample test cases. The integrated **Monaco Editor** allows users to write, execute, and test their code directly within the platform.

AI Learn and Chatbot Assistance:

The **AI Learn** feature provides detailed editorials, step-by-step approaches, and optimized code solutions using **Gemini API**.

Additionally, the built-in **AI Chatbot** offers real-time, context-aware guidance — helping users debug errors and clarify problem logic through natural conversation.

Progress Tracking:

All problem attempts, AI interactions, and results are stored in **MongoDB Atlas**. Users can track their learning history and performance over time through their dashboard.

Technical Output:

Frontend:

Developed using **ReactJS** and **Tailwind CSS**, the interface is responsive, user-friendly, and optimized for cross-device compatibility. Component-based design and React Hooks ensure maintainability and scalability.

Backend:

Built with **Node.js**, **Express.js**, and **MongoDB**, the backend manages authentication, data handling, and API communication with the Gemini service. RESTful APIs ensure seamless interaction between frontend, database, and AI modules.

Educational and Practical Impact:

CodeClash.AI enhances programming skills through hands-on coding, AI-guided learning, and contextual feedback. It serves as a **virtual coding mentor**, helping students build logic, debug effectively, and prepare for technical interviews.

Instructors can also utilize it as a learning aid to monitor student progress and assign targeted practice problems.

10. Output

Figure 10.1 - Login Page

The screenshot shows the CodeClash login page. The header includes the CodeClash logo, navigation links for 'Explore', 'Problems', and 'Topics', and buttons for 'Sign In' and 'Sign Up'. The main content area is split into two columns. The left column features the CodeClash logo, the heading 'Master coding with AI-powered insights', a subtext 'Join 50,000+ developers practicing on our platform with real-time feedback and multi-language support.', and three bullet points: 'AI-powered code analysis', 'Collaborative learning', and 'Competitive programming'. The right column is titled 'Welcome' and contains the text 'Sign in to explore more'. It features a 'Continue with Google' button, a link 'Or continue with email', an 'Email address' input field with the placeholder 'Enter your email', a 'Password' input field with the placeholder 'Enter your password' and a toggle for visibility, a 'Remember me' checkbox, a 'Forgot password?' link, and a 'Sign In' button with a right arrow.

Figure 10.2 - Sign Up Page

The screenshot shows the CodeClash sign up page. The header is identical to the login page. The main content area is split into two columns. The left column features the CodeClash logo, the heading 'Join the coding revolution', a subtext 'Start your journey with AI-powered learning, real-time feedback, and a community of passionate developers.', and a section titled 'Why developers choose CodeClash:' with two bullet points: 'AI-Powered Insights' (Get instant feedback and optimization suggestions) and 'Learn Together' (Join study groups and collaborative coding sessions). The right column is titled 'Create your account' and contains the text 'Join thousands of developers'. It features a 'Continue with Google' button, a link 'Or create with email', a 'Full Name' input field with the placeholder 'Enter your full name', an 'Email address' input field with the placeholder 'Enter your email', and a 'Password' input field with the placeholder 'Create a password' and a toggle for visibility.

Figure 10.3 - Landing Page



Figure 10.4 - Problem List Interface

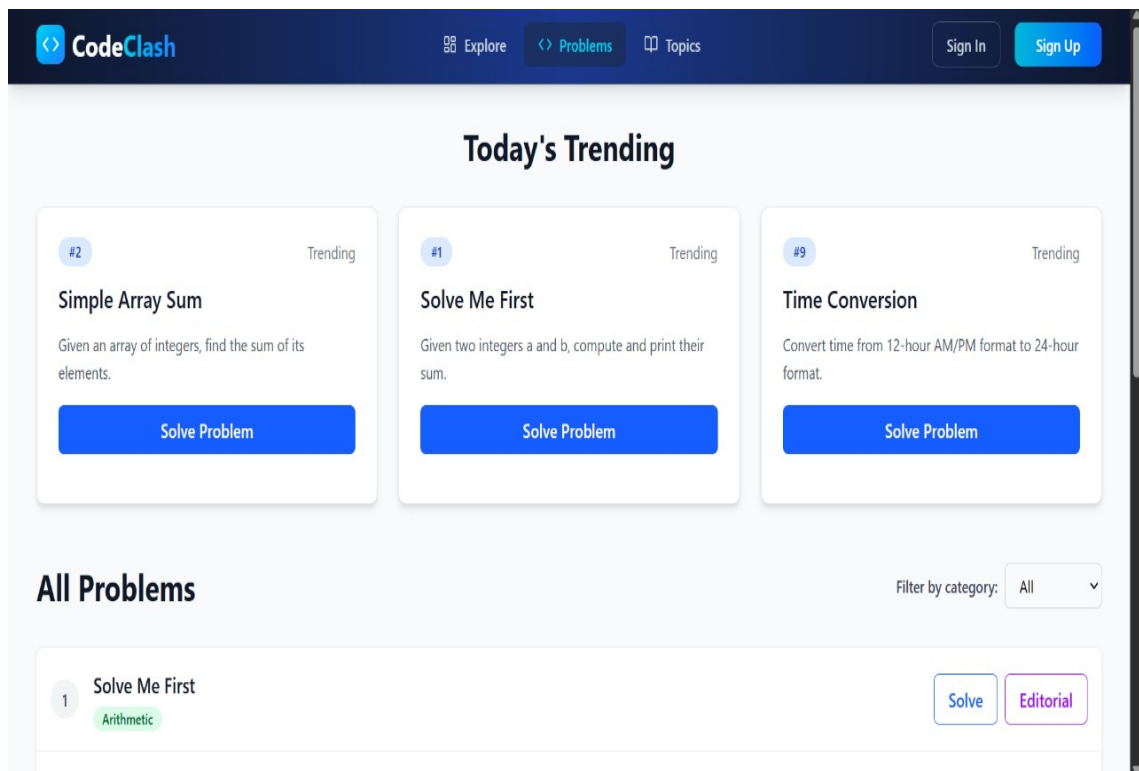


Figure 10.5 - Code Editor

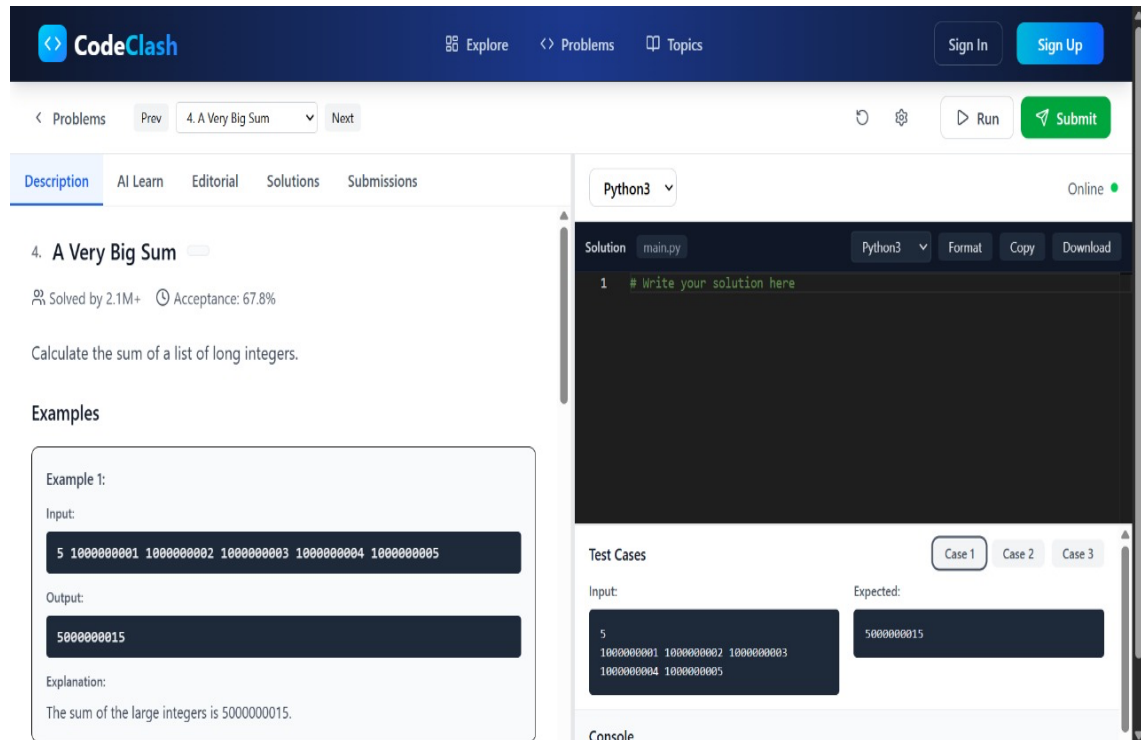


Figure 10.6 - AI Learn Panel

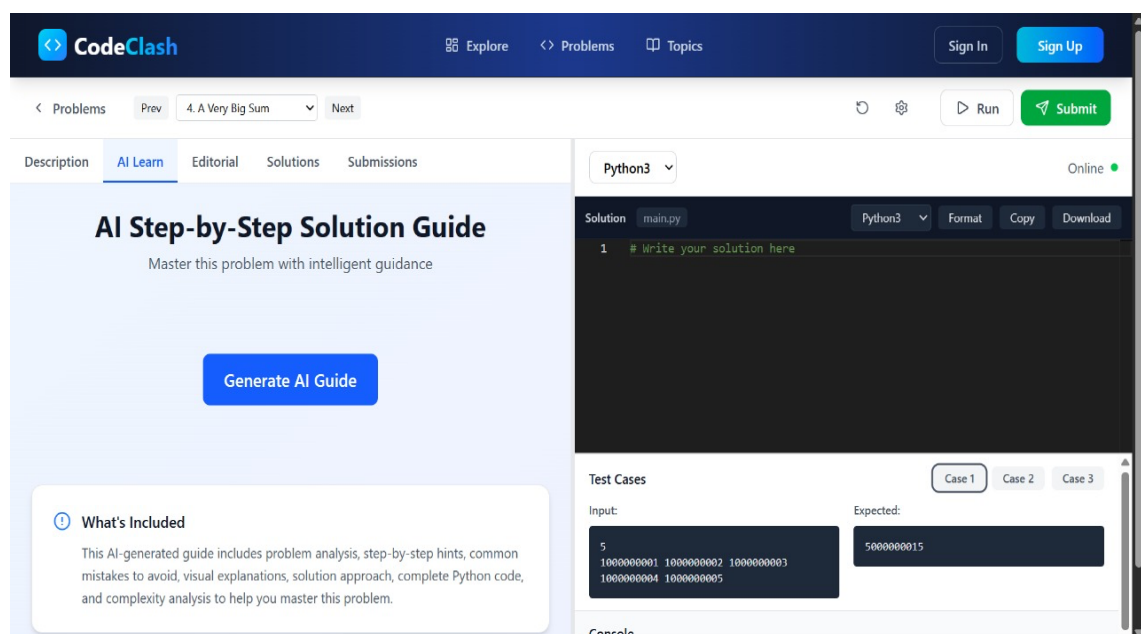


Figure 10.7 - AI Chatbot Interface

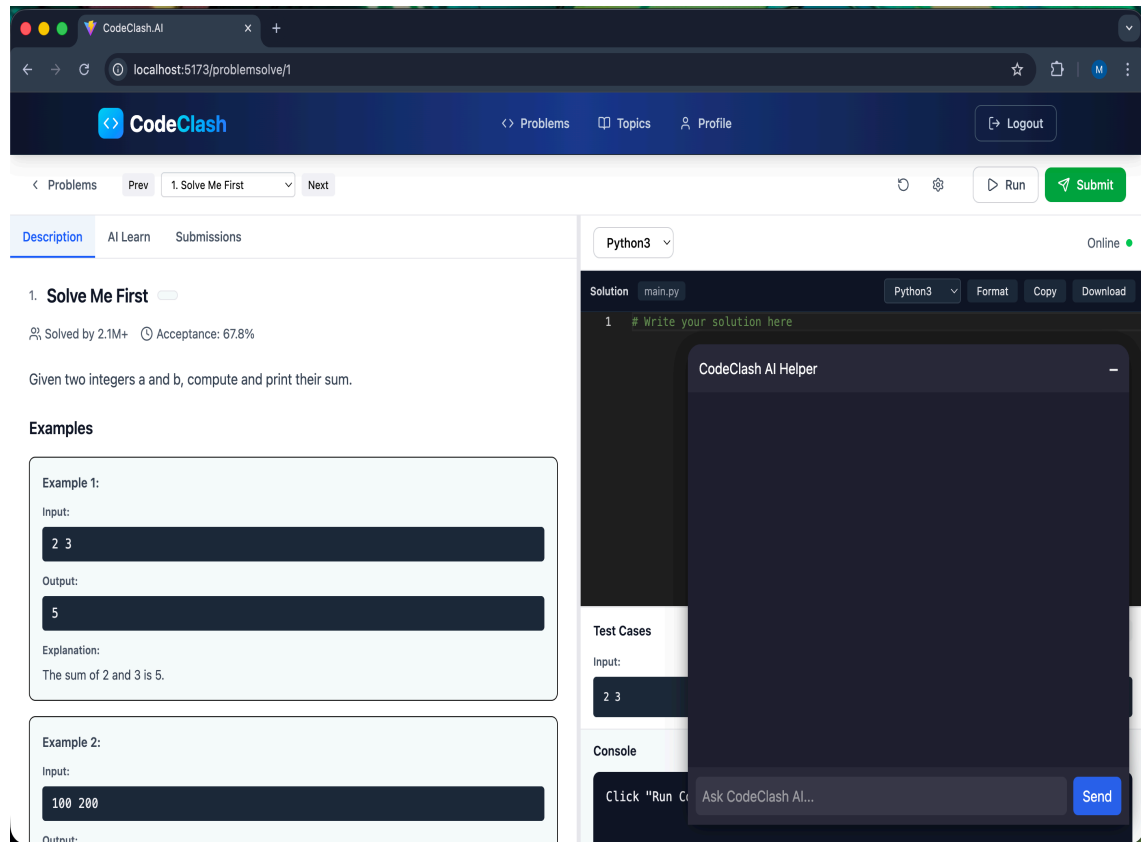


Figure 10.8 - Code Execution Display

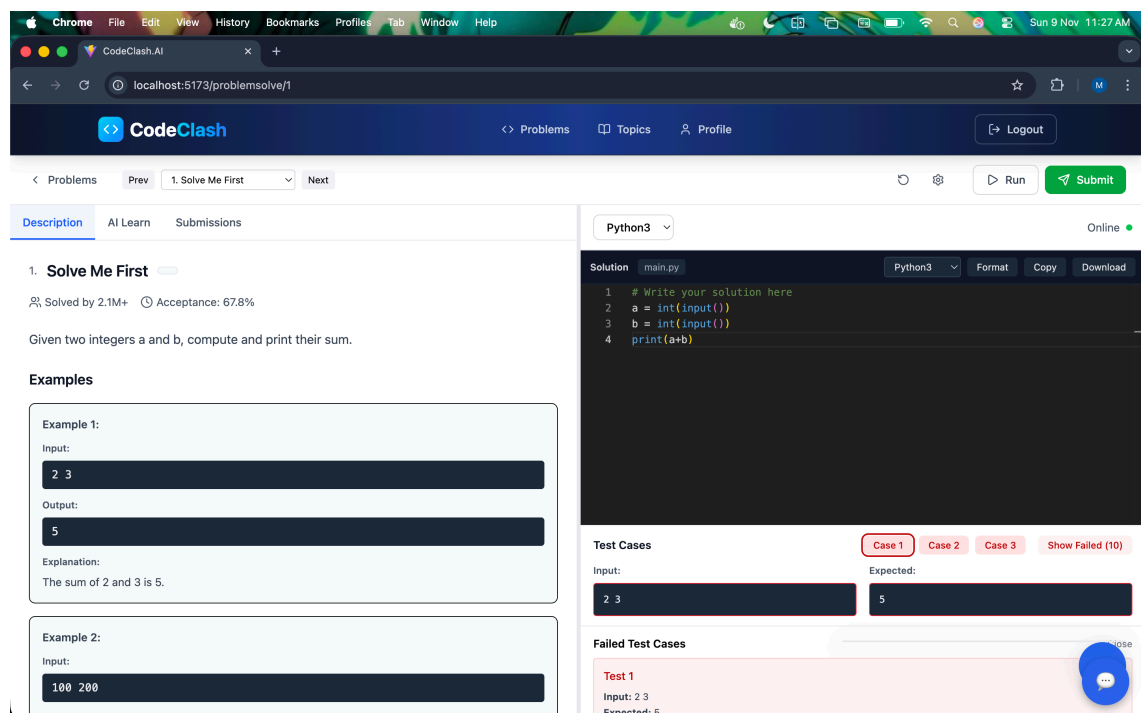


Figure 10.9 - Profile Page

The screenshot shows a web browser window with the CodeClash application running on localhost:5173. The browser tabs include 'CodeClash.AI', 'Mayank2553 (Mayank Srivastava)', and 'Mayank Srivastava | LinkedIn'. The application's navigation bar has links for 'Problems', 'Topics', 'Profile' (active), and 'Logout'.

The profile page for 'Mayank Srivastava' (monty123@gmail.com) features a blue header bar with a profile picture placeholder and a 'Save Changes' button. Below the header, the page is divided into three main sections:

- About:** A text area with the placeholder 'Tell us about yourself...'.
- Personal Information:** A form with fields for Date of Birth (05/05/2025), Gender (Male), Country (India), and College/University (KIET Group of Institutions).
- Coding Stats:** A section showing '0 Problems Solved'.
- Social Links:** A section with two links: 'https://github.com/Mayank255' and 'https://www.linkedin.com/in/mayank255'.

11. Conclusion

CodeClash.AI successfully provides an **AI-powered coding and learning platform** that transforms the way students practice programming and understand algorithms.

By integrating modern web technologies such as **ReactJS**, **Node.js**, **Express.js**, and **MongoDB**, the system delivers a seamless and interactive learning experience for users.

The platform enables users to solve coding problems in a real-time editor and gain conceptual clarity through **AI-driven learning assistance** powered by the **Gemini API**. Features such as **AI Learn** for editorial explanations and **AI Chatbot** for contextual guidance make CodeClash.AI a comprehensive tool for both skill development and conceptual understanding.

Throughout the development process, the focus remained on **security, scalability, and user experience**. JWT-based authentication ensures secure user sessions, while MongoDB Atlas provides reliable cloud data storage. The platform underwent multiple testing phases — including unit, integration, functional, and security testing — to ensure stability and performance across environments.

In conclusion, CodeClash.AI successfully bridges the gap between **theory and practical learning**, offering an intelligent and adaptive environment for students to code, learn, and improve continuously.

Future enhancements may include:

- AI-based personalized problem recommendations,
- Adaptive learning difficulty,
- Support for multiple programming languages, and
- Integration with learning management systems (LMS) for classroom use.

With its strong foundation and scalable architecture, CodeClash.AI represents a forward-thinking step toward **AI-assisted education and self-paced coding mastery**.

12. References

1. **Node.js Official Documentation** – <https://nodejs.org/en/docs>
2. **Express.js Framework Guide** – <https://expressjs.com/en/starter/installing.html>
3. **MongoDB Manual (NoSQL Database)** – <https://www.mongodb.com/docs/manual/>
4. **MongoDB Atlas Cloud Database** – <https://www.mongodb.com/atlas/database>
5. **ReactJS Official Documentation** – <https://react.dev/learn>
6. **React Hooks Documentation** – <https://react.dev/reference/react>
7. **Tailwind CSS Documentation** – <https://tailwindcss.com/docs>
8. **Monaco Editor (VS Code Editor) Documentation** – <https://microsoft.github.io/monaco-editor/>
9. **Axios HTTP Client Documentation** – <https://axios-http.com/docs/intro>
10. **JWT Authentication – Auth0 Documentation** – <https://auth0.com/docs/secure/tokens/json-web-tokens>
11. **bcrypt Password Hashing Documentation** – <https://www.npmjs.com/package/bcrypt>
12. **Google Gemini API Documentation** – <https://ai.google.dev/docs>
13. **Prompt Engineering Guide for LLMs** – <https://platform.openai.com/docs/guides/prompt-engineering> (*Used conceptually for AI prompt design*)
14. **Postman API Testing Guide** – <https://learning.postman.com/docs/getting-started/introduction/>
15. **RESTful API Design Guidelines** – <https://restfulapi.net/>
16. **OWASP Web Application Security Top 10** – <https://owasp.org/www-project-top-ten/>
17. **Docker Documentation (Code Execution Isolation)** – <https://docs.docker.com/get-started/>
18. **React Toastify – Notification Library** – <https://www.npmjs.com/package/react-toastify>
19. **JavaScript MDN Documentation** – <https://developer.mozilla.org/en-US/docs/Web/JavaScript>